

Reviewer Pack — Schur-Weyl Complexity of Tseitin Expanders

Entry fc7bf27d129b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 13:02:53 UTC

Schur-Weyl Complexity of Tseitin Expanders

Entry ID: fc7bf27d129b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 13:02:53 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Tseitin Formulas on Expander Graphs

Statement:

For a Tseitin formula derived from an expander graph G with n vertices, the number of irreducible components in the Schur-Weyl decomposition of the tensor product of G 's adjacency matrix with itself is $\Omega(2^n)$.

Rationale (proposer's reasoning):

Expander graphs' high connectivity induces complex tensor structures that require more irreducible components in their decomposition, potentially correlating with the proof complexity of the Tseitin formula via representation-theoretic invariants.

Taxonomy category: GCT_DET_PERM (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3d9b814a9ccdd8ca

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.5s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_expander_graph(n):
        # Generate a random expander graph using the Chordal Ring construction
        graph = {i: set() for i in range(n)}
        for i in range(n):
            j = (i + 1) % n
            k = (i - 1) % n
            graph[i].add(j)
            graph[j].add(i)
            graph[i].add(k)
            graph[k].add(i)
        return graph

    def adjacency_matrix(graph, n):
        mat = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in graph[i]:
                mat[i][j] = 1
        return mat

    def tensor_product(mat1, mat2):
        n = len(mat1)
        result = [[0] * (n * n) for _ in range(n * n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    for l in range(n):
                        result[i * n + k][j * n + l] += mat1[i][j] * mat2[k][l]
        return result

    def count_irreducible_components(mat, n):
        # Placeholder function to count irreducible components
        # This is a dummy implementation and should be replaced with actual computation
        return 0

    n = random.randint(5, 40)
    graph = generate_expander_graph(n)
    mat = adjacency_matrix(graph, n)
    tensor_mat = tensor_product(mat, mat)
    irreducible_count = count_irreducible_components(tensor_mat, n)
```

```

metric_value = irreducible_count
instances_tested = 1
conjecture_holds = False
counterexample = "mapping_undefined"

return {
    "metric_name": "irreducible_components",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 67, 71, 73, 79, 83, 89, 97]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'irreducible_components', 'metric_value': 0, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4eae02df.py", line 90, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4eae02df.py", line 90, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    ~^^^^^^^^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing valid data; no reliable results to assess conjecture. | next: Implement robust error handling for metric collection and re-run with concrete expander instances

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	103523
2	preregistration	ollama_remote	qwen3:8b	0	0	27576
3	novelty	ollama_remote	qwen3:8b	0	0	27541
4	novelty	ollama_remote	qwen3:8b	0	0	19888
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13927
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10480
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9927
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9694
9	judge	ollama_remote	qwen3:8b	0	0	21267

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 243822 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/fc7bf27d129b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fc7bf27d129b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/fc7bf27d129b.tar.gz` (if generated)